

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ

Anul 2023 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Definitivat Model 2023	2
SUBIECTUL I (60 de puncte)	2
1. Algoritmul de sortare prin Inserție (Insertion Sort)	2
a) Descriere în limbaj natural și exemplificare pe 7 numere	2
b) Complexitatea algoritmului	2
c) Problemă completă (Sortare prin inserție)	2
2. Rețele: Aplicații colaborative	3
3. Programare: Permutare circulară cifre	3
4. Baze de date: Curse de cai	5
II. Rezolvare Definitivat Varianta 3 (19 iulie 2023)	6
SUBIECTUL I (60 de puncte)	6
1. Algoritmul lui Dijkstra (Grafuri Ponderate)	6
a) Noțiuni preliminare:	6
b) Descrierea algoritmului Dijkstra pe un graf cu 7 noduri	6
c) Problemă completă (Dijkstra)	7
2. Tehnologia Informației (TIC): Formatarea la nivel de pagină	8
3. Programare: Cifra de control	9
4. Baze de date: Organizație de ciclism	10
SUBIECTUL al II-lea - Completare pentru Model 2023 (30 de puncte)	11
1. Strategie didactică pentru structuri de date	11
2. Itemi de completare	12
SUBIECTUL al II-lea - Completare pentru Varianta 3 2023 (30 de puncte)	12
1. Strategie didactică pentru secvența A: operații asupra datelor	12
2. Întrebări structurate	12

I. Rezolvare Definitivat Model 2023

SUBIECTUL I (60 de puncte)

1. Algoritmul de sortare prin Inserție (Insertion Sort)

a) Descriere în limbaj natural și exemplificare pe 7 numere

- **Descriere:** Algoritmul pornește de la al doilea element al tabloului și îl consideră pe rând drept „cheie”. Elementul cheie este comparat cu elementele din stânga sa (deja sortate) și deplasat spre stânga până când își găsește poziția corectă (când se întâlnește un element mai mic sau egal cu el).
- **Exemplificare pentru tabloul:** [35, 12, 45, 8, 20, 15, 30] ($N = 7$)
 1. **Pasul 1** (cheie = 12): $12 < 35$, deplasăm 35. Tabloul devine: [12, 35, 45, 8, 20, 15, 30].
 2. **Pasul 2** (cheie = 45): $45 > 35$, rămâne pe loc. Tabloul: [12, 35, 45, 8, 20, 15, 30].
 3. **Pasul 3** (cheie = 8): $8 < 45$, $8 < 35$, $8 < 12$. Deplasăm toate elementele. Tabloul: [8, 12, 35, 45, 20, 15, 30].
 4. **Pasul 4** (cheie = 20): $20 < 45$, $20 < 35$, $20 > 12$. Deplasăm 45 și 35. Tabloul: [8, 12, 20, 35, 45, 15, 30].
 5. **Pasul 5** (cheie = 15): $15 < 45$, $15 < 35$, $15 < 20$, $15 > 12$. Deplasăm 45, 35, 20. Tabloul: [8, 12, 15, 20, 35, 45, 30].
 6. **Pasul 6** (cheie = 30): $30 < 45$, $30 < 35$, $30 > 20$. Deplasăm 45 și 35. Tabloul final sortat: [8, 12, 15, 20, 30, 35, 45].

b) Complexitatea algoritmului

- **Cazul cel mai nefavorabil (Worst case):** Vectorul este sortat descrescător. Pentru fiecare element de la poziția i facem i comparații. Complexitatea este $O(N^2)$.
- **Cazul cel mai favorabil (Best case):** Vectorul este deja sortat crescător. Facem o singură comparație pentru fiecare cheie. Complexitatea este $O(N)$.
- **Cazul mediu (Average case):** Complexitatea timpului este tot de ordinul $O(N^2)$.

c) Problemă completă (Sortare prin inserție)

- **Enunț:** Să se ordoneze crescător elementele unui vector folosind sortarea prin inserție.
- **Cod C++:**

```
#include <iostream>
using namespace std;
void insertionSort(int A[], int n) {
    for (int i = 1; i < n; ++i) {
        int cheie = A[i];
        int j = i - 1;
        while (j >= 0 && A[j] > cheie) {
            A[j + 1] = A[j];
            j--;
        }
        A[j + 1] = cheie;
    }
}
int main() {
    int A[] = {35, 12, 45, 8, 20, 15, 30};
    insertionSort(A, 7);
    for (int x : A) cout << x << " ";
}
```

```
    return 0;
}
```

• **Cod Pascal:**

```
program SortareInsertie;
var
  A: array[1..7] of integer = (35, 12, 45, 8, 20, 15, 30);
  i, j, cheie: integer;
begin
  for i := 2 to 7 do
  begin
    cheie := A[i];
    j := i - 1;
    while (j >= 1) and (A[j] > cheie) do
    begin
      A[j + 1] := A[j];
      j := j - 1;
    end;
    A[j + 1] := cheie;
  end;
  for i := 1 to 7 do write(A[i], ' ');
end.
```

—

2. Rețele: Aplicații colaborative

- **Aplicații colaborative:** Programe ce permit utilizatorilor aflați în locații diferite să lucreze în timp real la același document sau proiect (ex. Microsoft Teams, Google Docs).
- **Dispozitive periferice utilizate:** Cameră web, microfon, căști/boxe.
- **Beneficii:** Reducerea costurilor de deplasare, flexibilitatea programului de lucru.
- **Dezavantaje:** Dependența critică de conexiunea la internet, riscuri crescute privind securitatea datelor.

—

3. Programare: Permutare circulară cifre

Soluție C++:

```
#include <iostream>
#include <fstream>
using namespace std;

void circular(long long &n) {
  long long temp = n;
  long long p = 1;
  while (temp >= 10) {
    p *= 10;
    temp /= 10;
  }
  int last = n % 10;
  n = last * p + (n / 10);
}

int main() {
  ifstream fin("def2022.in");
  long long x;
```

```

long long min_odd = -1;
int count_min = 0;
while (fin >> x) {
    circular(x);
    if (x % 2 != 0) {
        if (min_odd == -1 || x < min_odd) {
            min_odd = x;
            count_min = 1;
        } else if (x == min_odd) {
            count_min++;
        }
    }
}
fin.close();

if (min_odd == -1) cout << "nu exista\n";
else cout << min_odd << " " << count_min << "\n";

return 0;
}

```

Soluție Pascal:

```

program PermutareCifre;
var
    fin: text;
    x, min_odd: int64;
    count_min: integer;

procedure circular(var n: int64);
var
    temp, p: int64;
    last: integer;
begin
    temp := n;
    p := 1;
    while temp >= 10 do
        begin
            p := p * 10;
            temp := temp div 10;
        end;
    last := n mod 10;
    n := last * p + (n div 10);
end;

begin
    assign(fin, 'def2022.in');
    reset(fin);
    min_odd := -1;
    count_min := 0;
    while not eof(fin) do
        begin
            read(fin, x);
            circular(x);
            if x mod 2 <> 0 then
                begin
                    if (min_odd = -1) or (x < min_odd) then

```

```

begin
    min_odd := x;
    count_min := 1;
end
else if x = min_odd then
    count_min := count_min + 1;
end;
end;
close(fin);

if min_odd = -1 then writeln('nu exista')
else writeln(min_odd, ' ', count_min);
end.

```

4. Baze de date: Curse de cai

Model conceptual:

- CAL: id_cal, nume, varsta, nume_proprietar, telefon_proprietar, email_proprietar.
- CURSA: id_cursa, tip_cursa, data_start, ora_start, minut_start, status (propusa/desfasurata).
- INSCRIERE: asociază un cal cu o cursă și reține a_participat, loc_clasare.

Relații:

- CAL 1:M INSCRIERE; un cal poate fi înscris la mai multe curse.
- CURSA 1:M INSCRIERE; o cursă poate avea mai mulți cai înscriși.
- Relația M:N dintre cai și curse este normalizată în tabela INSCRIERE.

Reguli 1NF-3NF și restricții:

- **1NF**: datele de contact sunt câmpuri atomice, nu liste într-un singur atribut.
- **2NF**: în INSCRIERE, rezultatul depinde de întreaga cheie (id_cal, id_cursa).
- **3NF**: datele cursei nu se repetă la fiecare cal, iar datele calului nu se repetă la fiecare cursă.
- loc_clasare poate fi NULL dacă nu a participat; dacă a_participat = false, atunci loc_clasare trebuie să fie NULL.

Model fizic:

```

CREATE TABLE CAL (
    id_cal INT PRIMARY KEY AUTO_INCREMENT,
    nume VARCHAR(100) NOT NULL,
    varsta INT CHECK (varsta > 0),
    nume_proprietar VARCHAR(120) NOT NULL,
    telefon_proprietar VARCHAR(30),
    email_proprietar VARCHAR(120)
);

CREATE TABLE CURSA (
    id_cursa INT PRIMARY KEY AUTO_INCREMENT,
    tip_cursa VARCHAR(50) NOT NULL,
    data_start DATE NOT NULL,
    ora_start INT CHECK (ora_start BETWEEN 0 AND 23),
    minut_start INT CHECK (minut_start BETWEEN 0 AND 59),
    status VARCHAR(20) NOT NULL
);

```

```
CREATE TABLE INSCRIERE (
    id_cal INT,
    id_cursa INT,
    a_participat BOOLEAN DEFAULT FALSE,
    loc_clasare INT NULL,
    PRIMARY KEY (id_cal, id_cursa),
    FOREIGN KEY (id_cal) REFERENCES CAL(id_cal),
    FOREIGN KEY (id_cursa) REFERENCES CURSA(id_cursa)
);
```

SQL pentru ștergerea cailor care nu au mai fost înscriși în ultimii doi ani:

```
DELETE FROM CAL
WHERE id_cal NOT IN (
    SELECT DISTINCT i.id_cal
    FROM INSCRIERE i
    JOIN CURSA c ON i.id_cursa = c.id_cursa
    WHERE c.data_start >= DATE_SUB(CURDATE(), INTERVAL 2 YEAR)
);
```

II. Rezolvare Definitivat Varianta 3 (19 iulie 2023)

SUBIECTUL I (60 de puncte)

1. Algoritmul lui Dijkstra (Grafuri Ponderate)

a) Noțiuni preliminare:

- **Graf ponderat:** Un graf în care fiecărei muchii/arc i se asociază un număr real numit pondere sau cost.
- **Drum:** O succesiune de noduri în care fiecare două noduri consecutive sunt unite printr-o muchie/arc.
- **Cost al unui drum:** Suma ponderilor muchiilor/arcelor componente.

b) Descrierea algoritmului Dijkstra pe un graf cu 7 noduri

Fie graful orientat cu nodurile $\{1, 2, 3, 4, 5, 6, 7\}$ și nodul de start 1. Ponderile arcelor sunt: $(1, 2) : 2, (1, 3) : 4, (2, 3) : 1, (2, 4) : 7, (3, 5) : 3, (4, 6) : 1, (5, 4) : 2, (5, 7) : 5, (6, 7) : 2$.

Etapele algoritmului Dijkstra:

1. Inițializăm vectorul de distanțe d cu ∞ pentru toate nodurile, cu excepția nodului de start 1, pentru care $d[1] = 0$. Inițializăm o mulțime de noduri selectate $S = \emptyset$.
2. La fiecare pas i :
 - Alegem un nod u neselectat cu $d[u]$ minim.
 - Îl adăugăm în S .
 - Pentru fiecare vecin v al lui u , relaxăm arcul (u, v) : $d[v] = \min(d[v], d[u] + \text{cost}(u, v))$.
3. Repetăm pasul 2 de $N - 1$ ori.

Trace-ul algoritmului pentru graful ales:

- **Inițial:** $d = (0, \infty, \infty, \infty, \infty, \infty, \infty)$
- **Pas 1:** Selectăm nodul 1. Vecini: 2, 3.
 - $d[2] = \min(\infty, 0 + 2) = 2$
 - $d[3] = \min(\infty, 0 + 4) = 4$
 - $d = (0, 2, 4, \infty, \infty, \infty, \infty)$

- **Pas 2:** Selectăm nodul 2. Vecini: 3, 4.
 - $d[3] = \min(4, 2 + 1) = 3$
 - $d[4] = \min(\infty, 2 + 7) = 9$
 - $d = (0, 2, 3, 9, \infty, \infty, \infty)$
- **Pas 3:** Selectăm nodul 3. Vecin: 5.
 - $d[5] = \min(\infty, 3 + 3) = 6$
 - $d = (0, 2, 3, 9, 6, \infty, \infty)$
- **Pas 4:** Selectăm nodul 5. Vecini: 4, 7.
 - $d[4] = \min(9, 6 + 2) = 8$
 - $d[7] = \min(\infty, 6 + 5) = 11$
 - $d = (0, 2, 3, 8, 6, \infty, 11)$
- **Pas 5:** Selectăm nodul 4. Vecin: 6.
 - $d[6] = \min(\infty, 8 + 1) = 9$
 - $d = (0, 2, 3, 8, 6, 9, 11)$
- **Pas 6:** Selectăm nodul 6. Vecin: 7.
 - $d[7] = \min(11, 9 + 2) = 11$ (rămâne neschimbat)
 - $d = (0, 2, 3, 8, 6, 9, 11)$
- **Pas 7:** Selectăm nodul 7. Distanțele finale sunt $d = (0, 2, 3, 8, 6, 9, 11)$.

c) Problemă completă (Dijkstra)

- **Enunț:** Să se determine drumul de cost minim de la nodul de start 1 la toate celelalte noduri într-un graf ponderat cu n noduri și m arce.

• Cod C++:

```
#include <iostream>
#include <vector>
using namespace std;
const int INF = 1000000000;
int w[100][100], d[100];
bool viz[100];
int main() {
    int n, m, u, v, c;
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) {
        d[i] = INF; viz[i] = false;
        for (int j = 1; j <= n; ++j) w[i][j] = INF;
    }
    for (int i = 0; i < m; ++i) {
        cin >> u >> v >> c; w[u][v] = c;
    }
    d[1] = 0;
    for (int pas = 1; pas <= n; ++pas) {
        int min_val = INF, u = -1;
        for (int i = 1; i <= n; ++i)
            if (!viz[i] && d[i] < min_val) { min_val = d[i]; u = i; }
        if (u == -1) break;
        viz[u] = true;
        for (int v = 1; v <= n; ++v)
            if (w[u][v] != INF && d[u] + w[u][v] < d[v]) d[v] = d[u] + w[u][v];
    }
    for (int i = 1; i <= n; ++i) cout << d[i] << " ";
}
```

```
    return 0;
}
```

• **Cod Pascal:**

```
program Dijkstra;
const INF = 1000000000;
var
  n, m, u, v, c, i, j, pas, min_val, min_u: integer;
  w: array[1..100, 1..100] of integer;
  d: array[1..100] of integer;
  viz: array[1..100] of boolean;
begin
  read(n, m);
  for i := 1 to n do
  begin
    d[i] := INF; viz[i] := false;
    for j := 1 to n do w[i, j] := INF;
  end;
  for i := 1 to m do
  begin
    read(u, v, c); w[u, v] := c;
  end;
  d[1] := 0;
  for pas := 1 to n do
  begin
    min_val := INF; min_u := -1;
    for i := 1 to n do
      if (not viz[i]) and (d[i] < min_val) then
      begin
        min_val := d[i]; min_u := i;
      end;
    if min_u = -1 then break;
    viz[min_u] := true;
    for v := 1 to n do
      if (w[min_u, v] <> INF) and (d[min_u] + w[min_u, v] < d[v]) then
        d[v] := d[min_u] + w[min_u, v];
    end;
    for i := 1 to n do write(d[i], ' ');
  end.
```

2. Tehnologia Informației (TIC): Formatarea la nivel de pagină

- **Noțiuni preliminare:** Un document text este compus din secțiuni, pagini, paragrafe și caractere. Elemente de interfață într-un procesor de text: Rigla (ruler), bara de stare, meniul de configurare.
- **Personalizare pagină:**
 1. **Margini:** Ajustarea spațiului gol de pe margini (Top, Bottom, Left, Right, Gutter).
 2. **Orientare:** Portret (vertical) sau Peisaj (orizontal).
 3. **Dimensiune hârtie:** Ex. A4, Letter, A3.
 4. **Coloane:** Divizarea textului paginii pe mai multe coloane.
 5. **Antet și subsol (Header & Footer):** Zona destinată titlurilor sau numerotării.
 6. **Fundal pagină:** Culori de fundal sau filigrane (watermarks).

3. Programare: Cifra de control

- **Descriere:** Subprogramul `suma(n)` calculează suma cifrelor unui număr natural. Algoritmul din programul principal procesează elementele din fișierul `def2023.in`, le determină cifra de control în mod repetat și contorizează minimul obținut.

Soluție C++:

```
#include <iostream>
#include <fstream>
using namespace std;

int suma(long long n) {
    int s = 0;
    while (n > 0) {
        s += n % 10;
        n /= 10;
    }
    return s;
}

int cifraControl(long long n) {
    if (n == 0) return 0;
    long long s = n;
    while (s >= 10) {
        s = suma(s);
    }
    return s;
}

int main() {
    ifstream fin("def2023.in");
    long long x;
    int min_cifra = 10;
    int count_min = 0;
    while (fin >> x) {
        int cc = cifraControl(x);
        if (cc < min_cifra) {
            min_cifra = cc;
            count_min = 1;
        } else if (cc == min_cifra) {
            count_min++;
        }
    }
    fin.close();
    cout << min_cifra << " " << count_min << endl;
    return 0;
}
```

Soluție Pascal:

```
program CifraDeControl;
var
    fin: text;
    x: int64;
    min_cifra, count_min, cc: integer;

function suma(n: int64): integer;
```

```

var s: integer;
begin
    s := 0;
    while n > 0 do
    begin
        s := s + (n mod 10);
        n := n div 10;
    end;
    suma := s;
end;

function cifraControl(n: int64): integer;
var s: int64;
begin
    if n = 0 then cifraControl := 0
    else
    begin
        s := n;
        while s >= 10 do
            s := suma(s);
        cifraControl := s;
    end;
end;

begin
    assign(fin, 'def2023.in');
    reset(fin);
    min_cifra := 10;
    count_min := 0;
    while not eof(fin) do
    begin
        read(fin, x);
        cc := cifraControl(x);
        if cc < min_cifra then
        begin
            min_cifra := cc;
            count_min := 1;
        end
        else if cc = min_cifra then
            count_min := count_min + 1;
        end;
    close(fin);
    writeln(min_cifra, ' ', count_min);
end.

```

4. Baze de date: Organizație de ciclism

Model conceptual:

- COMPETITIE: id_competitie, denumire, oras_secretariat, telefon_contact, email_contact, site_web.
- TIP_PROBA: id_tip_proba, denumire_tip, descriere_generala.
- PROBA: id_proba, id_competitie, id_tip_proba, descriere_particulara, data_desfasurare, categorie_participanti.

Relații:

- COMPETITIE 1:M PROBA; o competiție poate conține mai multe probe.
- TIP_PROBA 1:M PROBA; un tip de probă poate apărea în mai multe competiții.
- Separarea TIP_PROBA evită repetarea descrierii generale pentru probe de același tip.

Reguli 1NF-3NF și restricții:

- **1NF**: datele de contact, categoria și data sunt valori atomice.
- **2NF**: toate atributele non-cheie depind complet de cheia primară a tabelului.
- **3NF**: descrierea generală a tipului de probă depinde de id_tip_proba, nu de id_proba.
- email_contact trebuie să fie unic sau validat la nivel aplicație; data_desfasurare este obligatorie pentru probe planificate/desfășurate.

Model fizic:

```
CREATE TABLE COMPETITIE (  
    id_competitie INT PRIMARY KEY AUTO_INCREMENT,  
    denumire VARCHAR(150) NOT NULL,  
    oras_secretariat VARCHAR(100) NOT NULL,  
    telefon_contact VARCHAR(30),  
    email_contact VARCHAR(120),  
    site_web VARCHAR(150)  
);  
  
CREATE TABLE TIP_PROBA (  
    id_tip_proba INT PRIMARY KEY AUTO_INCREMENT,  
    denumire_tip VARCHAR(60) NOT NULL UNIQUE,  
    descriere_generala TEXT  
);  
  
CREATE TABLE PROBA (  
    id_proba INT PRIMARY KEY AUTO_INCREMENT,  
    id_competitie INT NOT NULL,  
    id_tip_proba INT NOT NULL,  
    descriere_particulara TEXT,  
    data_desfasurare DATE NOT NULL,  
    categorie_participanti VARCHAR(60) NOT NULL,  
    FOREIGN KEY (id_competitie) REFERENCES COMPETITIE(id_competitie),  
    FOREIGN KEY (id_tip_proba) REFERENCES TIP_PROBA(id_tip_proba)  
);
```

SQL pentru afișarea denumirilor competițiilor cu cel puțin o probă în anul curent:

```
SELECT DISTINCT c.denumire  
FROM COMPETITIE c  
JOIN PROBA p ON c.id_competitie = p.id_competitie  
WHERE YEAR(p.data_desfasurare) = YEAR(CURDATE());
```

—

SUBIECTUL al II-lea - Completare pentru Model 2023 (30 de puncte)

1. Strategie didactică pentru structuri de date

Metodă: exercițiul dirijat. **Avantaje**: fixează operațiile elementare pe structuri de date și permite feedback imediat asupra erorilor de parcurgere, inserare sau ștergere.

Mijloc: mediu de programare și schemă vizuală a listei înlănțuite. **Formă**: frontal, apoi lucru individual. **Activitate**: inserarea unui nod într-o listă simplu înlănțuită.

Scenariu: Profesorul reprezintă nodurile și legăturile, cere elevilor să indice ordinea actualizării pointerilor, apoi elevii implementează operația. Se verifică științific faptul că legătura noului nod se setează înainte de a pierde adresa succesorului.

2. Itemi de completare

- **Caracteristici:** răspuns scurt, corectare obiectivă, integrare într-un enunț incomplet. **Reguli:** enunț clar, un singur răspuns așteptat, spațiul liber să nu elimine contextul.
- **Item A:** O coadă este o structură de date de tip [spațiu liber]. **Răspuns:** FIFO.
- **Item B:** Memoria folosită pentru stocarea temporară a datelor în timpul funcționării calculatorului este [spațiu liber]. **Răspuns:** memoria RAM.

SUBIECTUL al II-lea - Completare pentru Varianta 3 2023 (30 de puncte)

1. Strategie didactică pentru secvența A: operații asupra datelor

Formă de organizare: activitate frontală urmată de lucru individual. **Justificare:** profesorul poate fixa terminologia operațiilor aritmetice, logice și relaționale, iar elevii aplică imediat conceptele în exerciții.

Mijloc: fișă de lucru și mediu de programare. **Metodă:** conversația euristică. **Activitate:** evaluarea unor expresii aritmetice, relaționale și logice.

Scenariu: Profesorul scrie expresii precum $a+b*2$, $x<10$, $(x>0)$ and $(x<100)$, elevii stabilesc ordinea operațiilor și tipul rezultatului, apoi verifică prin program.

2. Întrebări structurate

Caracteristici: au context comun și cerințe gradate; permit evaluarea mai multor operații cognitive. **Reguli:** cerințe clare, punctaj pe subcerințe, răspuns așteptat explicit.

Item A: Se dau $a=7$, $b=3$.

1. Calculați $a \div b$.
2. Calculați $a \bmod b$.
3. Precizați valoarea expresiei $(a>b)$.

Răspuns: 2; 1; adevărat.

Item B: Despre unitatea centrală:

1. Numiți o componentă a UCP.
2. Precizați rolul acesteia.
3. Dați un exemplu de parametru al microprocesorului.

Răspuns: ALU - efectuează operații aritmetice/logice; exemplu parametru: frecvența, numărul de nuclee, memoria cache.